

Technical Blog Manager

Mini-Project - Console Based, Menu Driven Application

App Type	Console application - Menu driven program
Programming Language	C++
Data Management	File I/O
Team	Team of 2

Project Idea

Develop a console-based application to maintain small technical blog posts. The user can create, view, edit, delete, and search blogs, and can organize blogs into multiple custom lists such as Favorites, Java, Security, or any other user-defined list. All information should be stored using file I/O.

Functional Requirements

1. The application should allow the user to manage mini technical blogs using CRUD operations - Add, Display, Update/Edit, and Delete.
2. Each blog should contain the following fields: Blog ID, Title, Date, and Content.
3. Date may be automatically assigned by the application or entered by the user, as per the implementation choice of the team.
4. Blog content should be plain text. No formatting or Markdown support is required.
5. The application should provide search functionality by Blog ID and Title.
6. The application should allow the user to create custom lists. Example lists: Favorites, Java, Security, Interview Tips, etc.
7. Custom list names should be unique. The application should provide operations to Create, Display, and Delete lists.
8. A blog can be added to one or more custom lists using the Blog ID.
9. The application should provide operations to Add Blog to List, Remove Blog from List, and View Blogs in a selected List.
10. Deleting a list should delete only the list. Blogs associated with that list must remain available in the application.
11. Deleting a blog should also remove that blog from all custom lists.
12. All blogs, lists, and their associations must be persisted using files so that data is available after application restart.
13. The application should validate basic input such as duplicate IDs, duplicate list names, invalid Blog IDs, and invalid menu choices.

Blog Data Fields

Field	Description	Example
Blog ID	Unique identifier for the blog	101
Title	Title of the technical blog	Understanding Java Interfaces
Date	Blog creation/publication date	24-09-2026
Content	Plain text technical content	An interface defines a contract that implementing classes follow...

Custom List Data

Field	Description	Example
List ID	Unique identifier for the custom list	1
List Name	Unique user-defined name	Java
Blog IDs	Blog IDs associated with this list; a blog may occur in multiple lists	101, 105, 110

Important Points to Note

- This is a single-user application; registration, login, logout, and user management are not required.
- This is a console-based menu-driven application.
- All application data must be stored and retrieved using files. No database is required.
- Blog IDs must be unique and custom list names must be unique.
- A blog may belong to zero, one, or multiple custom lists.
- Deleting a custom list must not delete any blog.
- Deleting a blog must remove its Blog ID from every custom list containing it.
- Search by Blog ID and Title is sufficient; advanced search is not required.
- Output should be well formatted and easy to understand for a live console demonstration.
- Refer the screenshots to get the clarity of implementation flow.

Evaluation Focus

Evaluation will focus on correct functionality, menu flow, file handling, validation, handling of blog-list relationships, code organization, and completeness of the required features. Students are encouraged to keep the implementation simple, reliable, and well formatted within the given time.

Happy Programming!!!

You can refer the following screenshots for more clarity.

```
=====
                        TECHNICAL BLOG & LIST MANAGER
=====
```

1. Blog Management
2. Custom List Management
3. Blog List Operations
4. Exit

Enter choice: 1

```
=====
                        BLOG MANAGEMENT
=====
```

1. Add Blog
2. Display All Blogs
3. View Blog Details
4. Edit Blog
5. Delete Blog
6. Search Blog
7. Back

Enter choice:
Invalid choice.

Press Enter to continue...

```
=====
                        BLOG MANAGEMENT
=====
```

1. Add Blog
2. Display All Blogs
3. View Blog Details
4. Edit Blog
5. Delete Blog
6. Search Blog
7. Back

Enter choice: 1

```
=====
                        ADD BLOG
=====
```

Blog ID: 101

Title: Semaphore

Date [25-09-2026]: 22-09-2026

Enter content. Press Enter on an empty line to finish.

A synchronization primitive with wait/signal operations used to control access to shared resources or coordinate processes/threads.

Blog added successfully.

Press Enter to continue...

```
=====
                        BLOG MANAGEMENT
=====

1. Add Blog
2. Display All Blogs
3. View Blog Details
4. Edit Blog
5. Delete Blog
6. Search Blog
7. Back

Enter choice: 2

=====
                        ALL BLOGS
=====
```

ID	Title	Date
101	Semaphore	22-09-2026
102	Memory Management	23-09-2026
103	I/O Management	24-09-2026
104	list(Python)	23-09-2026
105	structure(C Prog)	22-09-2026
106	Pointers(C Prog)	24-09-2026

```
-----
Press Enter to continue...
=====
```

```
=====
                        BLOG MANAGEMENT
=====

1. Add Blog
2. Display All Blogs
3. View Blog Details
4. Edit Blog
5. Delete Blog
6. Search Blog
7. Back

Enter choice: 3

=====
                        BLOG DETAILS
=====

Enter Blog ID: 101

Blog ID : 101
Title   : Semaphore
Date    : 22-09-2026
-----
A synchronization primitive with wait/signal operations used to control access to shared resources or coordinate processes/threads.
Press Enter to continue...
=====
```

```
=====
                        BLOG MANAGEMENT
=====

1. Add Blog
2. Display All Blogs
3. View Blog Details
4. Edit Blog
5. Delete Blog
6. Search Blog
7. Back

Enter choice: 4

=====
                        EDIT BLOG
=====

Enter Blog ID: 101
Press Enter to keep the existing value.
Title [Semaphore]:
Date [22-09-2026]:
Change content? (y/n): y
Enter content. Press Enter on an empty line to finish.
A binary semaphore acts like a mutex to enforce mutual exclusion, while a counting semaphore manages access to a pool o
f multiple identical resources.

Blog updated successfully.
```

Delete and Search Blog on the basis of Blog ID. (Screen shots not given of those)

```
=====
                        TECHNICAL BLOG & LIST MANAGER
=====

1. Blog Management
2. Custom List Management
3. Blog List Operations
4. Exit

Enter choice: 2

=====
                        CUSTOM LIST MANAGEMENT
=====

1. Create List
2. Display Lists
3. Delete List
4. Back

Enter choice: █
```

```

=====
                        TECHNICAL BLOG & LIST MANAGER
=====
1. Blog Management
2. Custom List Management
3. Blog List Operations
4. Exit

Enter choice: 2

=====
                        CUSTOM LIST MANAGEMENT
=====
1. Create List
2. Display Lists
3. Delete List
4. Back

Enter choice: 1

=====
                        CREATE LIST
=====

List Name: OS

List created successfully.

Press Enter to continue...

```

```

=====
                        CUSTOM LIST MANAGEMENT
=====
1. Create List
2. Display Lists
3. Delete List
4. Back

Enter choice: 2

=====
                        CUSTOM LISTS
=====
ID      List Name          Blogs
-----
1       OS                 0
2       Python             0
3       C Prog             0

Press Enter to continue...

```

Delete the list on the basis of list ID or name (screenshot is not given of it)

```

=====
                        TECHNICAL BLOG & LIST MANAGER
=====
1. Blog Management
2. Custom List Management
3. Blog List Operations
4. Exit

Enter choice: 3

=====
                        BLOG LIST OPERATIONS
=====
1. Add Blog to List
2. Remove Blog from List
3. View Blogs in List
4. Back

Enter choice:

```

```
=====
                        BLOG LIST OPERATIONS
=====

1. Add Blog to List
2. Remove Blog from List
3. View Blogs in List
4. Back

Enter choice: 1

=====
                        ADD BLOG TO LIST
=====

ID      List Name          Blogs
-----
1       OS                 0
2       Python             0
3       C Prog             0

Enter List ID: 1
Enter Blog ID to add: 101
Blog added to list successfully.

Press Enter to continue...
```

```
=====
                        BLOG LIST OPERATIONS
=====

1. Add Blog to List
2. Remove Blog from List
3. View Blogs in List
4. Back

Enter choice: 3

=====
                        VIEW BLOGS IN LIST
=====

ID      List Name          Blogs
-----
1       OS                 3
2       Python             0
3       C Prog             0

Enter List ID: 1

List: OS

-----
ID      Title               Date
-----
101     Semaphore           22-09-2026
102     Memory Management     23-09-2026
103     I/O Management         24-09-2026

Press Enter to continue...|
```

ALL BLOGS		
ID	Title	Date
101	Semaphore	22-09-2026
102	Memory Management	23-09-2026
103	I/O Management	24-09-2026
104	list(Python)	23-09-2026
105	structure(C Prog)	22-09-2026
106	Pointers(C Prog)	24-09-2026

For example,

here we added blog no 101,102,103 in list “OS”

Blog no 104 added in list “Python”

Blog no 105,106 added in list “C Prog”

BLOG LIST OPERATIONS		
1. Add Blog to List 2. Remove Blog from List 3. View Blogs in List 4. Back		
Enter choice: 3		
VIEW BLOGS IN LIST		
ID	List Name	Blogs
1	OS	3
2	Python	1
3	C Prog	2
Enter List ID:		

VIEW BLOGS IN LIST		
ID	List Name	Blogs
1	OS	3
2	Python	1
3	C Prog	2
Enter List ID: 2		
List: Python		
ID	Title	Date
104	list(Python)	23-09-2026
Press Enter to continue...		

```

=====
                        BLOG LIST OPERATIONS
=====
1. Add Blog to List
2. Remove Blog from List
3. View Blogs in List
4. Back

Enter choice: 3

=====
                        VIEW BLOGS IN LIST
=====
ID      List Name          Blogs
-----
1       OS                 3
2       Python             1
3       C Prog             2

Enter List ID: 3

List: C Prog

-----
ID      Title               Date
-----
105     structure( C Prog )  22-09-2026
106     Pointers( C Prog )   24-09-2026

Press Enter to continue...

```

```

=====
                        BLOG LIST OPERATIONS
=====
1. Add Blog to List
2. Remove Blog from List
3. View Blogs in List
4. Back

Enter choice: 2

=====
                        REMOVE BLOG FROM LIST
=====
ID      List Name          Blogs
-----
1       OS                 3
2       Python             1
3       C Prog             2

Enter List ID: 

```